Informe del SP4: integración con JavaScript

Lonera Carrera: lonería, y fabricación y alquiler de castillos inflables.

Nombre y apellido: Tomas Godoy	**Materia:** Aplicaciones Web I - Sistemas
Trabajo: SP4, integración con JavaScript	**Profesor:** Andrés Senn

Alcance. La consigna agrupa SP4 y SP5. En esta entrega se desarrolló todo el punto 4 de la consigna general (eventos, JSON, `fetch`, vistas desde JSON, validación, recursos en carpetas y modularización), así que esta entrega cubre ambos.

1. Qué se entregó

Las tres páginas del SP3 (`index.html`, `catalogo.html`, `contacto.html`) ahora funcionan: el catálogo se dibuja desde un archivo JSON, se filtra al instante, se arma un pedido que se conserva entre páginas y los formularios se validan antes de enviarse. El HTML y el CSS del SP3 se mantienen; el cambio principal es que las listas dejaron de estar escritas a mano en el HTML.

```
lonera-carrera/
├── index.html · catalogo.html · contacto.html · README.md
├── css/      variables · base · layout · componentes · paginas
├── data/     productos.json · reservas.json
├── fonts/ · img/
└── js/
    ├── comun.js · catalogo.js · contacto.js     Un punto de entrada por página
    ├── modulos/   api · almacen · carrito · filtros · disponibilidad · validacion
    │              pedidos · avisos · menu · dom · formato
    └── vistas/    lista-productos · pedido · ficha · confirmacion
```

2. Cómo se ejecuta

El código usa módulos (`<script type="module">`) y `fetch`. Ninguno de los dos funciona si la página se abre con doble clic (`file://`): el navegador los bloquea. Hay que servir la carpeta con un servidor local, por ejemplo la extensión *Live Server* de VS Code o `python -m http.server 8000` dentro de `lonera-carrera/`. Si alguien abre el archivo sin servidor, la página lo detecta y muestra un aviso que lo explica, y el menú queda visible aunque no corra JavaScript.

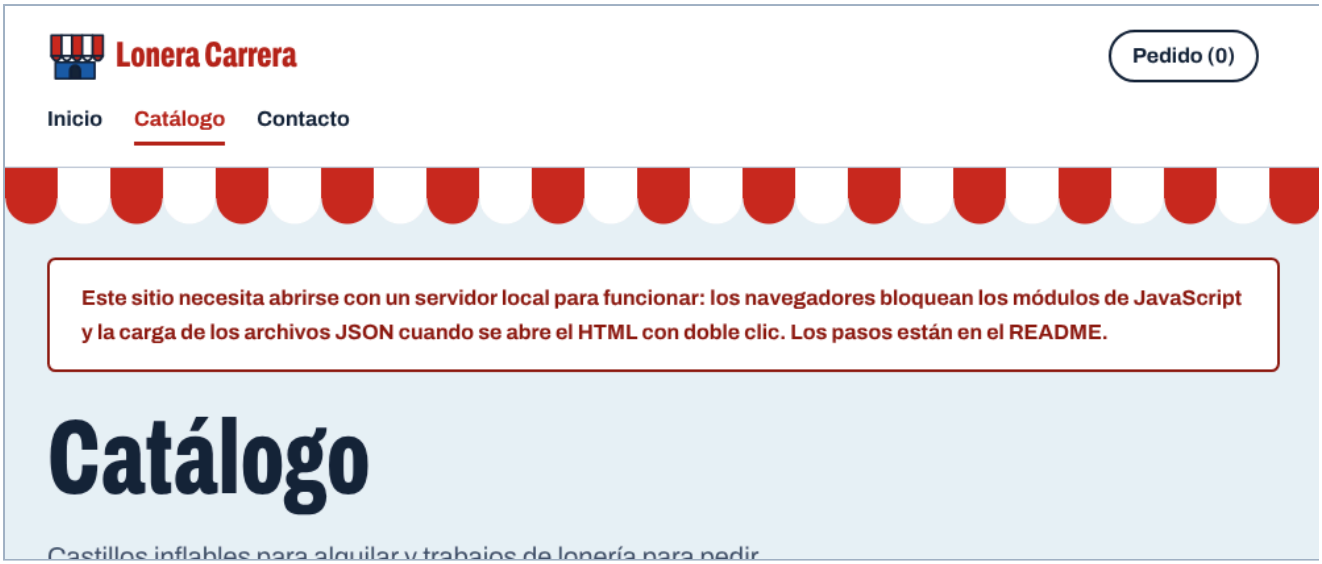


Figura 1. Aviso al abrir el catálogo con doble clic.

3. Arquitectura y modularización

Cada archivo hace una sola cosa. Los **módulos** no conocen la página (no buscan elementos por id, salvo avisos y menú); las **vistas** dibujan en el DOM; y cada página tiene un archivo de entrada que los conecta. Así la lógica (filtrar, validar, calcular el subtotal) se puede probar sin tocar el HTML.

Archivo	Responsabilidad
`modulos/api.js`	Único lugar donde se llama a `fetch`: `cargarCatalogo()` pide `productos.json` y `reservas.json` y verifica que la respuesta sea correcta.
`modulos/carrito.js`	El pedido: agregar, quitar, cambiar cantidad, vaciar, subtotal. Guarda en `sessionStorage` y avisa a quien se suscriba (cabecera y tabla).
`modulos/filtros.js`	Lee el formulario de filtros y filtra la lista (texto sin distinguir mayúsculas ni tildes, tipo, rangos).
`modulos/disponibilidad.js`	Decide si un castillo está libre en una fecha y qué texto y botón corresponden.
`modulos/validacion.js`	Reglas reutilizables y funciones para marcar errores con atributos ARIA.
`modulos/pedidos.js`	Envío simulado: número de pedido, guardado en `localStorage` y reservas locales.
`modulos/almacen.js`	Lee y escribe `sessionStorage`/`localStorage` con `try/catch` (si el navegador lo bloquea, la página sigue andando).
`modulos/avisos.js` · `menu.js` · `dom.js` · `formato.js`	Notificaciones, menú responsive, ayudas para crear elementos y formato de precio y fecha (es-AR).
`vistas/*`	Dibujan la lista de productos, la tabla del pedido, la ficha (`<dialog>`) y la confirmación.

`catalogo.js` · `contacto.js` · `comun.js`	Puntos de entrada: conectan eventos, módulos y vistas. `comun.js` (menú y contador) lo usan las tres páginas.

Todo el DOM se arma con `textContent` y `createElement` (nunca `innerHTML` con datos), de modo que un texto del JSON o del formulario no puede inyectar HTML.

4. Datos en JSON

`data/productos.json` reemplaza las tarjetas que estaban escritas en el HTML. Cada producto tiene el mismo formato; los campos que no aplican valen `null`.

```
{ "id": "castillo-clasico", "categoria": "castillos", "accion": "alquiler",
  "nombre": "Castillo clásico", "descripcion": "...", "imagen": "img/castillo-
clasico.svg",
  "alt": "...", "ancho": 320, "alto": 240,
  "precioPorDia": 45000, "capacidadMax": 8, "estadoPresupuesto": null,
  "detalles": [ { "etiqueta": "Medidas", "valor": "4 × 4 m" }, ... ] }
```

`data/reservas.json` lista las fechas ya ocupadas: `{ "reservas": [ { "productoId": "castillo-tobogan", "fecha": "2026-12-12" } ] }`. Hay 6 productos (3 castillos de alquiler, 1 castillo a medida y 2 trabajos de lonería) y 6 reservas de ejemplo. El archivo hace de "API" simulada: si más adelante existe un servidor real, solo cambia la dirección en `api.js`.

5. Eventos e interacción

Evento	Dónde	Qué pasa
`input`	Filtros (texto, tipo, capacidad, precio, fecha)	La lista se vuelve a dibujar al instante, con el resumen "Mostrando N productos" (o el mensaje de sin resultados).
`submit` / `reset`	Filtros	"Buscar" aplica los filtros; "Limpiar filtros" vuelve a mostrar los 6 productos.
`click` (delegado)	Lista de productos	Un único oyente en la lista maneja "Agregar al pedido" y "Ver ficha" de todas las tarjetas, aunque se redibujen.
`click` / `change`	Tabla del pedido	Quitar, cambiar cantidad (1 a 20) y vaciar el pedido.
`submit`	Pedido y contacto	Validación, envío simulado y confirmación.
`focusout` / `input`	Campos de formulario	Un campo con error se corrige y el mensaje desaparece mientras se escribe.
`click`, `keydown`, `pageshow`	Menú, ficha, cabecera	Botón Menú (`aria-expanded`), Escape cierra menú y ficha, y el contador se refresca al volver con "Atrás".

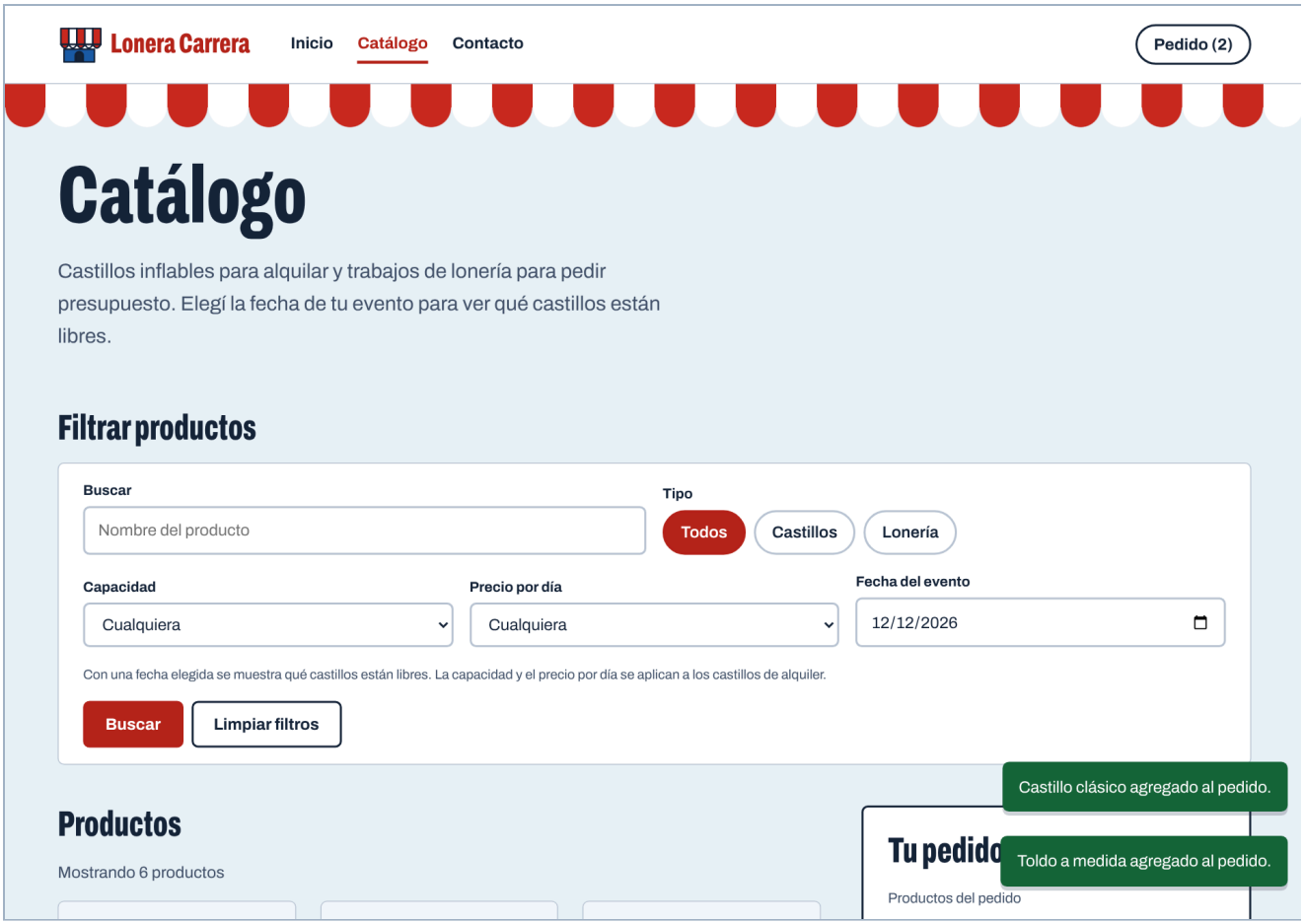


Figura 2. Catálogo con fecha del evento elegida, contador en la cabecera y avisos de "agregado al pedido".

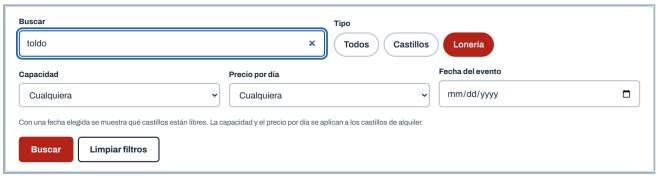


Figura 3. Filtros: tipo "Lonería" desde el enlace del inicio y búsqueda "toldo".

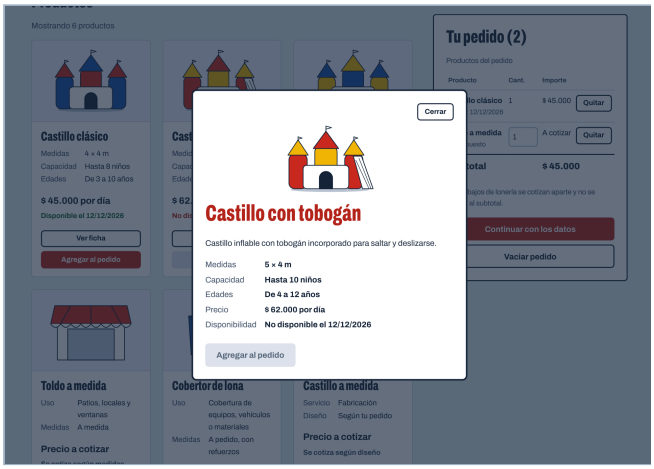


Figura 4. Ficha en `<dialog>`: el tobogán figura ocupado el 12/12/2026 y su botón está deshabilitado.

6. Disponibilidad y pedido

- Cada castillo se alquila una vez por fecha. El estado sale de cruzar `reservas.json` con las reservas de pedidos confirmados desde este navegador (`localStorage`).
- Sin fecha elegida se pide elegirla; con fecha, cada castillo dice "Disponible el 13/12/2026" o "No disponible el 12/12/2026". Un castillo ocupado tiene el botón deshabilitado (`No disponible`).
- Al confirmar, se vuelve a controlar la disponibilidad por si otra reserva ocupó la fecha mientras tanto. Después de enviar, esos castillos pasan a mostrarse como no disponibles esa fecha.

- El subtotal suma solo los castillos (precio por día); los trabajos de lonería se cotizan aparte y la tabla lo aclara.

7. Validación de formularios

Los formularios tienen `novalidate` para que los mensajes los controle el código (en español y junto a cada campo) en lugar de los globos del navegador. Cada campo tiene una lista de reglas; se muestra el primer error.

Campo	Reglas
Nombre y apellido	Obligatorio; al menos dos palabras, solo letras (acepta tildes y apóstrofos).
Teléfono	Obligatorio; 8 a 13 números. Se aceptan espacios, guiones, paréntesis y `+` (se ignoran al contar).
Correo	Obligatorio; formato `nombre@dominio.ext`.
Dirección (pedido)	Obligatoria solo si se elige "Envío a domicilio"; al volver a "Retiro" el error se borra.
Motivo (contacto)	Obligatorio.
Fecha (contacto)	Opcional; si se completa, desde hoy en adelante.
Mensaje (contacto)	Obligatorio; mínimo 10 caracteres.

Cada error es un `<p id="campo-error">` enlazado con `aria-describedby`, el campo recibe `aria-invalid="true"` y el foco va al primer campo con error. El botón pasa a "Enviando…" con `aria-busy` mientras se procesa, para evitar envíos dobles.

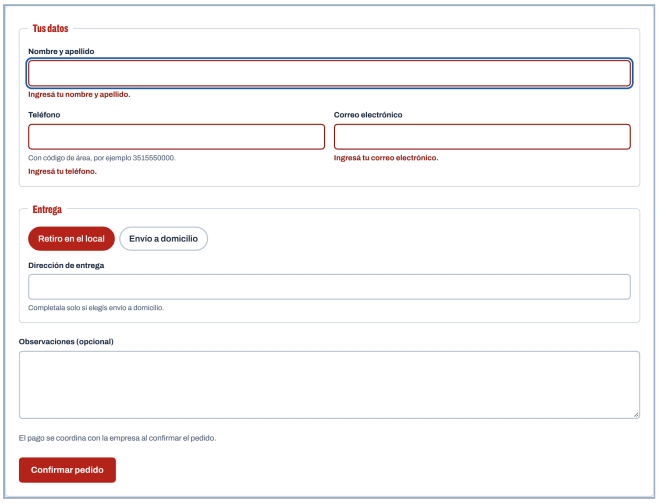


Figura 5. Formulario vacío al confirmar: tres errores y foco en el primero.



Figura 6. Confirmación con número de pedido, resumen y subtotal.

8. Notificaciones, errores y accesibilidad

- **Avisos (toasts)** dentro de una región `role="status"` que ya está en el HTML; los errores usan `role="alert"` y duran más. Se muestran como máximo 3, abajo, para no tapar el contenido.
- **Fallo de carga:** si `fetch` falla, el catálogo muestra un mensaje claro en lugar de quedar vacío, y lo mismo si falta el servidor.

- **Foco:** al quitar un producto el foco vuelve al título del pedido; al confirmar pasa a la confirmación; la ficha se cierra con Escape.
- **Sin JavaScript:** el menú queda visible (la clase `js` se agrega en el `<head>` solo si corresponde).



Figura 7. Móvil, menú cerrado.



Figura 8. Móvil, menú abierto con el botón.



Figura 9. Mensaje cuando no se puede cargar el catálogo (simulado bloqueando `productos.json`).

9. Requisitos de la consigna y dónde se cumplen

Requisito	Implementación
Eventos para filtros y formularios	`input`, `submit`, `reset`, `change`, `click` (sección 5).
Datos en JSON	`data/productos.json` y `data/reservas.json` (sección 4).
`fetch` global	`modulos/api.js`, con manejo de error.
Vista renderizada desde JSON	`vistas/lista-productos.js` (catálogo) y `vistas/pedido.js` (pedido).
Validación de formularios	`modulos/validacion.js` (sección 7).
Recursos en directorios	`css/`, `js/`, `data/`, `fonts/`, `img/`.
Modularización del código	Módulos ES en `js/modulos` y `js/vistas` (sección 3).

10. Cambios respecto de la propuesta

- **Sin servidor real.** Como esta materia todavía no incluye back-end, los datos viven en JSON estáticos y los pedidos y reservas nuevos se guardan en el navegador (`localStorage`). Por eso una reserva confirmada solo bloquea la fecha en el navegador donde se hizo. Con un servidor, ese guardado se reemplaza por una llamada `fetch` con `POST` en `pedidos.js`.
- **Envío simulado.** Se espera 0,7 s para imitar la red y se genera un número de pedido correlativo (`LC-0001`); no se envía ningún correo ni se cobra.
- **Filtros por eventos.** En el SP2 los filtros eran un formulario que enviaba la consulta y recargaba la página; ahora responden al instante, y se conservó el botón "Buscar" para quien lo use.
- **Pedido sin cuenta de usuario.** El pedido se guarda en la sesión del navegador (`sessionStorage`), no en una cuenta. Queda fuera lo que requeriría servidor: seguimiento del estado del pedido y la gestión interna de reservas.
- **Notificaciones.** Se sumaron avisos en pantalla que no estaban en la propuesta, para confirmar cada acción.

11. Cómo se verificó

- **Recorrido automatizado en el navegador** (Chromium, escritorio y 375 px de ancho): 74 comprobaciones, todas correctas. Incluyen filtros, disponibilidad por fecha, agregar/quitar, persistencia al recargar y entre páginas, ficha, errores de cada formulario, confirmación y guardado, fallo de `fetch`, menú móvil y apertura sin servidor.
- **Consola:** sin errores durante el recorrido (salvo el fallo de red provocado a propósito).
- **HTML:** validador W3C (Nu) sin errores ni advertencias en las tres páginas.
- **CSS:** stylelint con expresión regular BEM, sin incumplimientos.
- **Accesibilidad:** axe-core sin violaciones en las tres páginas, con la ficha abierta, con el formulario con errores, con la confirmación visible y con el menú móvil abierto.
- **Responsividad:** sin desborde horizontal a 375 px en inicio y catálogo.